

Internet of Things

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Reference book



Chapter 7

Outline

- ▶ API
- ▶ Prototyping Online Components
- ▶ GETTING STARTED WITH AN API
- ▶ MASHING UP APIS
- ▶ SCRAPING
- ▶ LEGALITIES
- ▶ References

Application programming interface

In computer programming, an ***application programming interface (API)*** is a set of routines, protocols, and tools for building software applications. An API expresses a software component in terms of its operations, inputs, outputs, and underlying types.



Web API

- **Web APIs** are the defined interfaces through which interactions happen between an enterprise and applications that use its assets. *Usually in Extensible Markup Language (**XML**) or JavaScript Object Notation (**JSON**) format.*
- **API** is typically defined as a set of *Hypertext Transfer Protocol (**HTTP**)* request messages.



Usability of web API

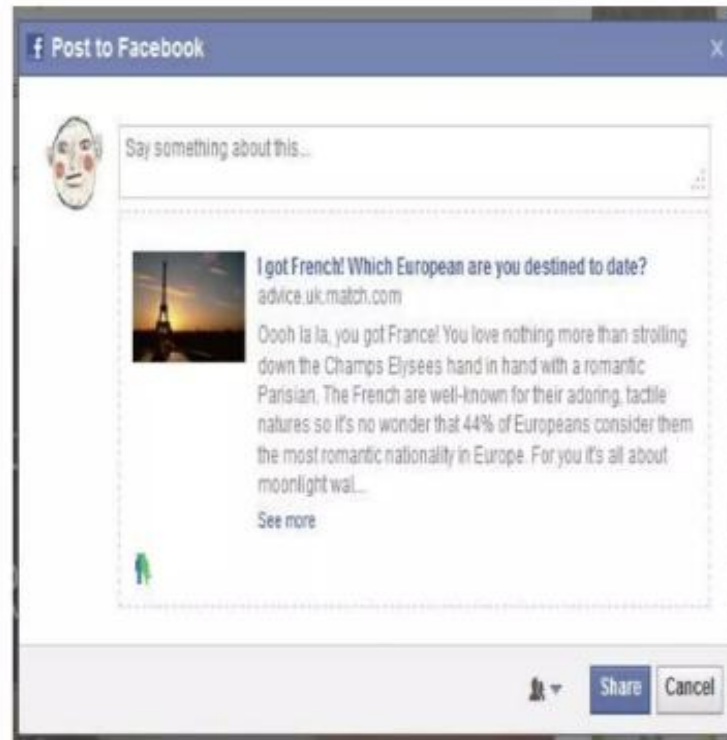
Photos can be shared from sites like Flickr and Photobucket to social network sites like Facebook and MySpace.



Share photos from Flickr

Usability of web API

Content can be dynamically posted. Sharing live comments made on Twitter with a Facebook account, for example, is enabled by their APIs. Etc.



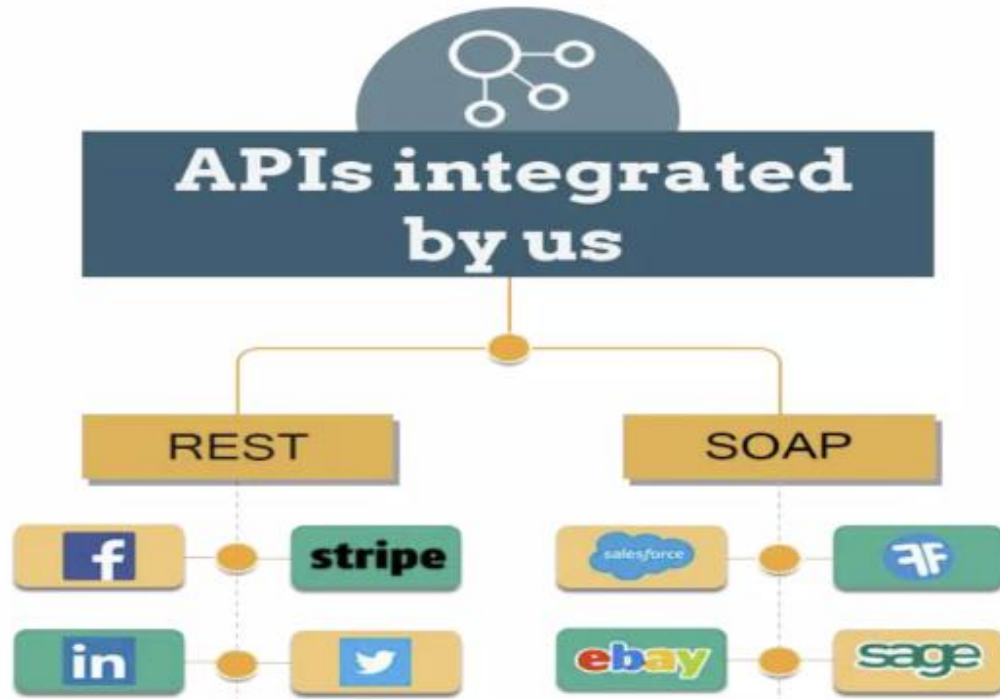
Web services

The two approaches for interfacing to the web with web services, namely:

- **SOAP (Simple Object Access Protocol)**
- **REST (Representational State Transfer)**

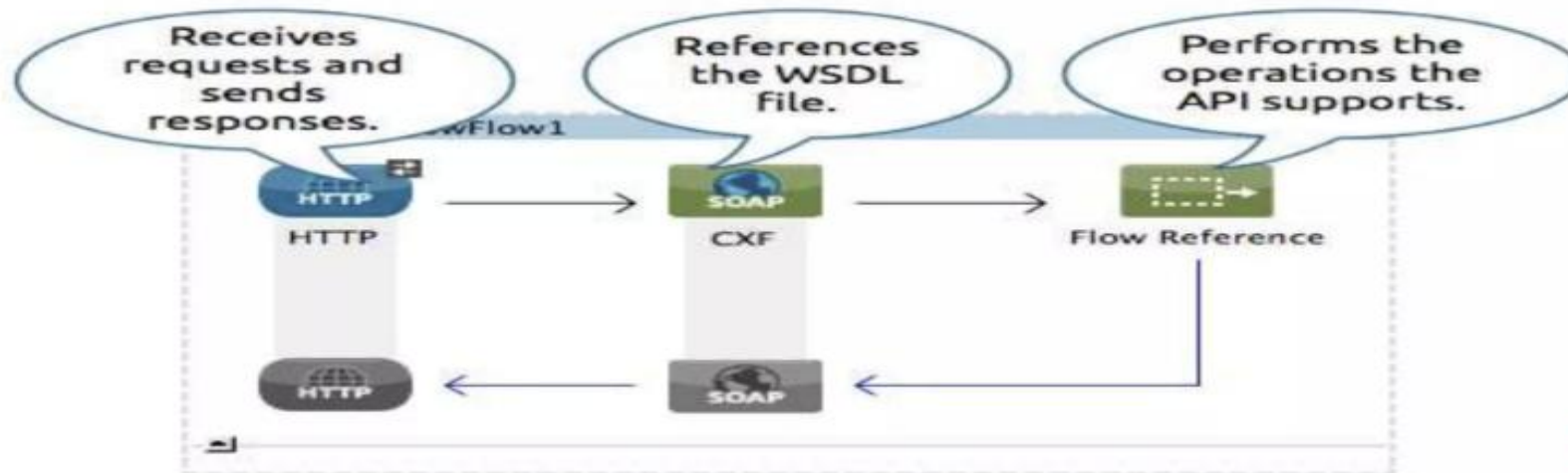


SOAP VS REST



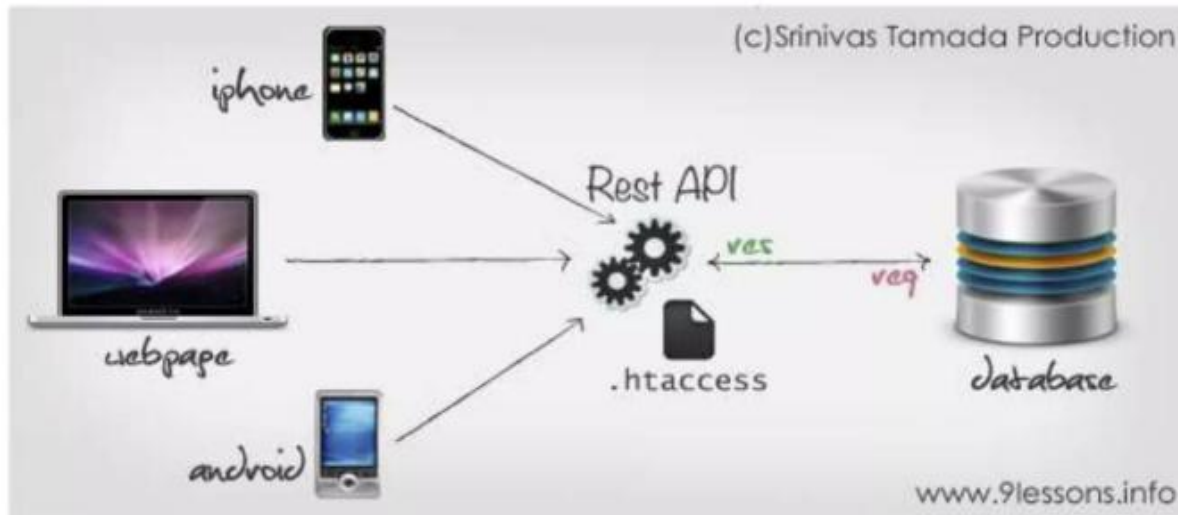
SOAP

Simple Object Access Protocol (SOAP) is a protocol for exchange of structured information on a decentralized and distributed platform using **XML (eXtensible Markup Language)**.



REST

Representational State Transfer or REST basically means that each unique URL is a representation of some object and supports format like **JSON, XML etc.**



Getting started with an API

- ▶ unlike a general-purpose device, such as a computer, tablet, or phone, the physical object is designed for a purpose and doesn't necessarily have a keyboard and screen to let it easily change its configuration
- ▶ Even with just these two components, you can already see some magical objects, such as air fresheners that pump fragrance into the room only when they sense that someone has walked past



Getting started with an API

- ▶ In the near future, devices will most likely use standardized protocols to speak to each other and to other apps and computers on your local or personal network.
- ▶ For now, though, in most of the examples we look at, each device is tied to a single web service. Although you've looked at existing services (Xively, Twitter), you might benefit from creating your own.
- ▶ For a personal project, creating such a service may not be important, but if you're developing a product to sell, you will want to be in control of **the service—otherwise, you may have to recall every device to reprogram any time it is discontinued, changes terms and conditions, making your use of it abusive, or changes its API, making your code stop working.**

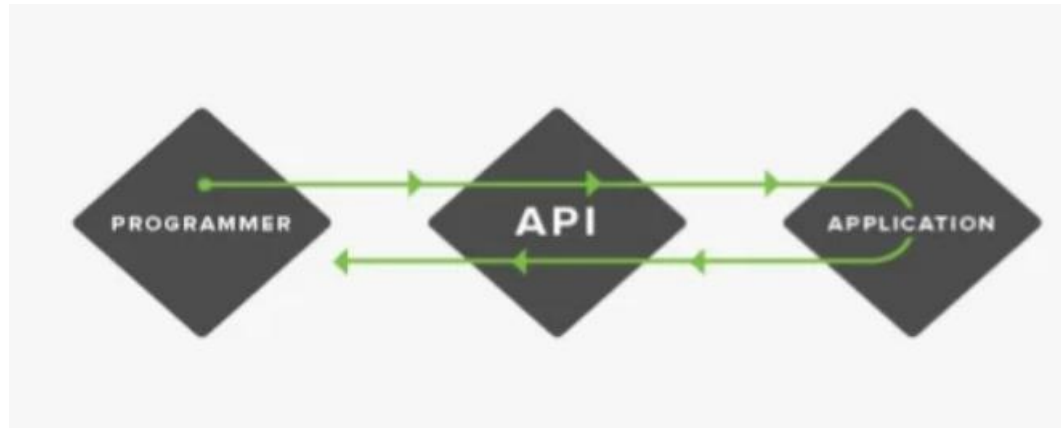
GETTING STARTED WITH AN API

- ▶ The most important part of a web service, with regards to an Internet of Things device, is the Application Programming Interface, or API.
- ▶ An API is a way of accessing a service that is targeted at machines rather than people.
- ▶ If you think about your experience of accessing an Internet service, you might follow a number of steps. For example, to look at a friend's photo on Flickr, you might want to share it in facebook or other website.



GETTING STARTED WITH AN API

- ▶ Although these actions are simple for a human, they involve a lot of looking, thinking, typing, and clicking. A computer can't look and think in the same way.
- ▶ Instead, a computer can very happily call defined commands such as login or get picture #142857



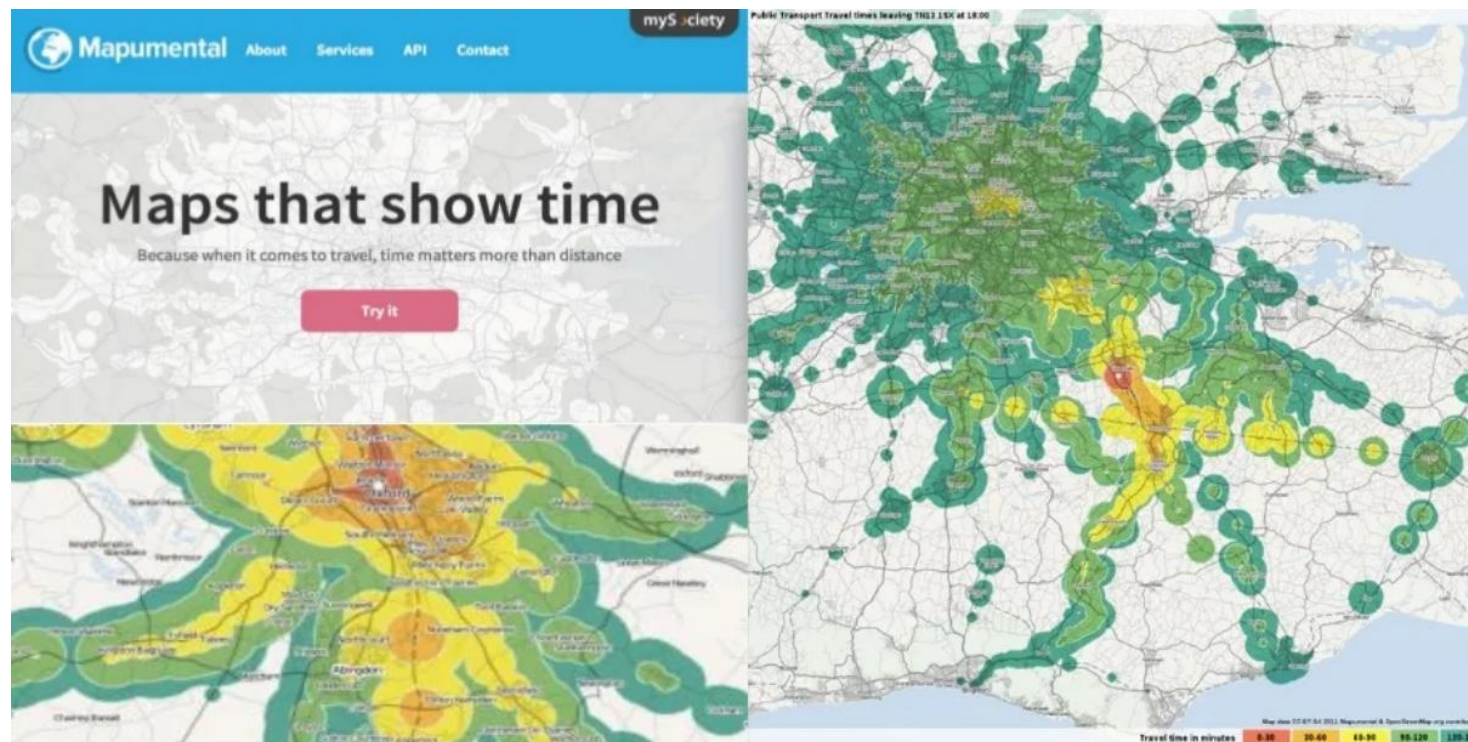
MASHING UP APIS

- ▶ Perhaps the data you want is already available on the Internet but in a form that doesn't work for you? The idea of “mashing up” multiple APIs to get a result has taken off and can be used to powerful effect. For example:
- ▶ Using a mapping API to plot properties to rent or buy—for example, Google Maps to visualize properties to rent via Craigslist, or Foxtons in London showing its properties using Mapumental.



MASHING UP APIS

- ▶ The Leading London estate agent Foxtons has become the first property player to use Mapumental maps on its website. Mapumental is a web-based application for displaying journey in terms of how long they take, rather than by distance.



MASHING UP APIS

- ▶ Fetching Flickr images that are related to the top headlines retrieved from The Guardian newspaper's API.

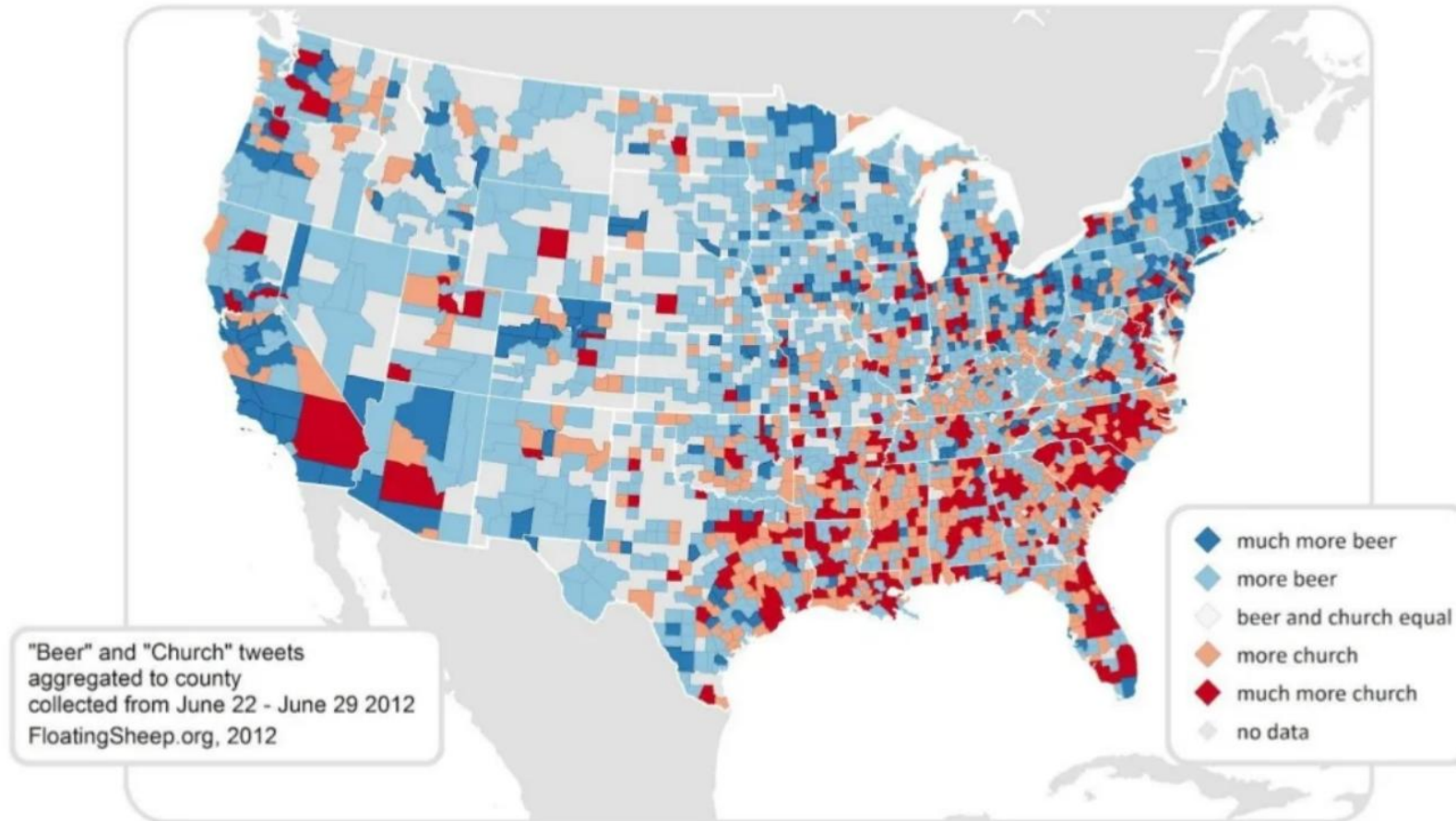


**OPEN
PLATFORM**
Build applications with **the guardian**



MASHING UP APIS

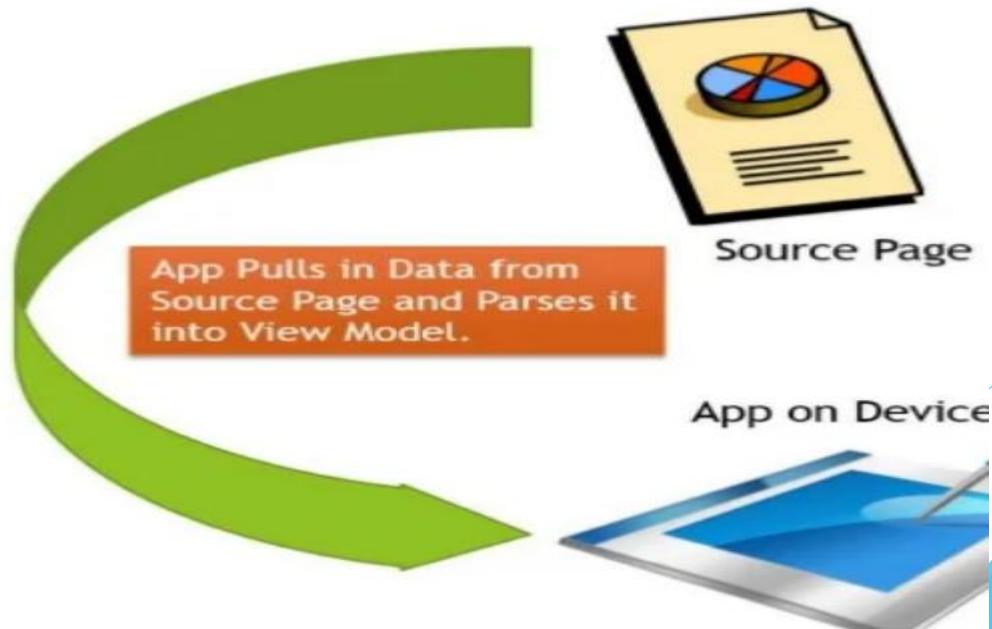
- ▶ Showing Twitter trends on a global map or in a timeline or a charting API.



SCRAPING

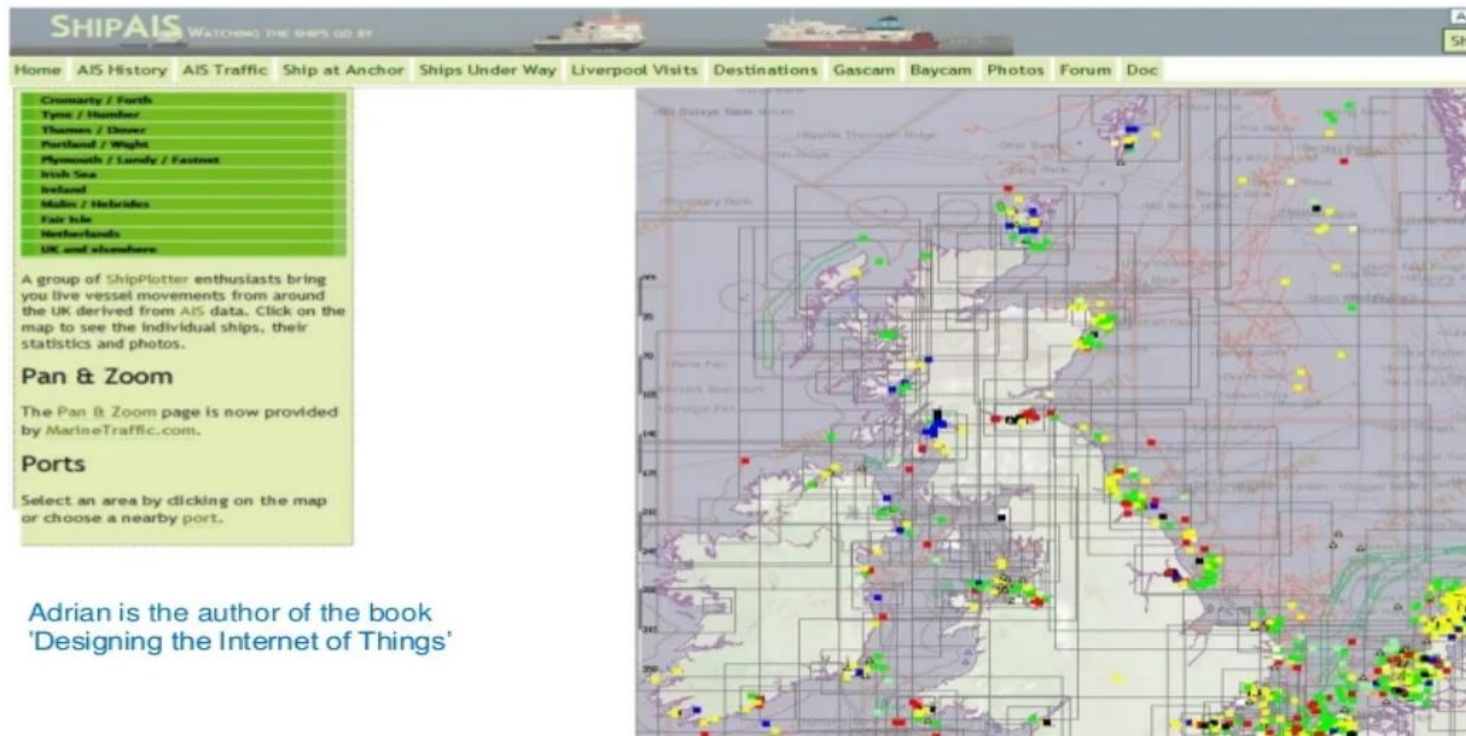
- ▶ Screen scraping is the process of collecting screen display data from one application and translating it so that another application can display it.
- ▶ In many cases, companies or institutions have access to fantastic data but don't want to or don't have the resources or knowledge to make them available as an API. In such case we use **screen scrapping**.

Basic Screen Scraping



SCRAPING

- ▶ Adrian has scraped **the Ship AIS system** (www.shipais.com/, whose data is semi-manually plotted by shipping enthusiasts) to get data about ships on the river Mersey, and this information is then tweeted by the @merseyshipping account (www.mcqn.com/weblog/connecting_river_mersey_twitter). He says of the project that it is a way to “connect the river to the Internet”,



Adrian is the author of the book
'Designing the Internet of Things'

Clicked on Berwick, to get information about ships and ports near to the area.

Tyne / Humber

- Cullercoats
- River Tyne
- Sunderland
- Scarborough
- River Humber
- Goole
- Skegness
- Thames / Dover
- Portland / Wight
- Plymouth / Lundy / Fastnet
- Irish Sea
- Ireland
- Malin / Hebrides
- Fair Isle
- Netherlands
- UK and elsewhere

Choose Map

Berwick

NW	N	NE
W	O	E
SW	S	SE

No refresh



SCRAPING

- ▶ The Public Whip website (www.publicwhip.org.uk/) is made possible by using a scraper to read the Hansard transcripts of UK government sessions (released as Word documents). With the resultant data, it can produce both human- and machine-readable feeds of how our elected representatives vote.



SCRAPING

- ▶ ScraperWiki site (<https://scraperwiki.com>) has an excellent platform for writing scrapers, in a number of dynamic programming languages, which collate data into database tables.
- ▶ Effectively, it provides infrastructure for “Mechanize” scripts that you could run on your own computer or server but allows you to outsource the boring, repetitive parts to ScraperWiki.
- ▶ Their CEO, Francis Irving, used this for an Internet of Things project of his own. He scrapes the Liverpool council website page to find out when his recycling bin is due to be collected.
- ▶ His Binduino device (<https://github.com/frabacus/binduino>), an Arduino plus custom electronics, checks the result regularly and illuminates some electroluminescent wire to make his recycling bin glow when he needs to take it out.



LEGALITIES

Screen-scraping may break the terms and conditions of a website. For example, Google doesn't allow you to screen-scrape its search pages but does provide an API.

Other data is protected by copyright or, for example, database rights. One project we discussed for the book would be a scraper that read football fixtures and moved a “compass” to point to the relative direction that your team was playing in. However, certainly in the UK, fixtures lists are copyrighted, and the English and Scottish football leagues have sued various operators for not paying them a licensing fee for that data (<http://www.out-law.com/page-10985>).

Alternative sources of information often are available. For example, you could use OpenStreetMap instead of Google Maps. The UK postcode database is under Crown Copyright, but there are other, perhaps partial, crowdsourced versions

Chapter -7: Prototyping Online Components

- ▶ Writing a New API
 - Clockodillo
 - Security
 - Implenting the API
 - Using c-URL to Test
- ▶ Real time Reactions
 - Polling
 - Comet
- ▶ **Other Protocols**
 - MQ Telemetry Transport

Writing a new API

- ▶ When you know what data you have, what actions can be taken on it, and what data will be returned, the **flows of your application became simple**.
- ▶ Although this might sound very different from writing a web application, it is actually an ideal way to start: by separating the business problem from the front end, you decouple the model (core data structure) from the view (HTML/JavaScript) and controller (widgets, form interaction and so on) .
- ▶ If you have programmed in **MVC(Model View Controllers)** frameworks (Ruby on Rails, Django, Catalyst, and so on) , it supports **development of friendly web pages or web applications** which is its advantage.
- ▶ The best news is, if you start designing an API in this way, you can easily add a website after words.

Writing A new API: Clockodillo

- ▶ Clockodillo is an [Internet-connected Task Timer](#).

The user can set a dial to a number of minutes, and the timer ticks down until completed. It also sends messages to an API server to let it know that a task has been started, completed, or cancelled.

A number of API interactions deal precisely with those features of the physical device:

- ❖ Start a new timer
- ❖ Change the duration of an existing timer
- ❖ Mark a timer completed
- ❖ Cancel a timer

Writing A new API: Clockodillo

Some interactions with a timer data structure are too complicated to be displayed on a device consisting mostly of a dial—for example, anything that might require a display or a keyboard! Those could be done through an app on your computer or phone instead.

- ❖ View and edit the timer's name/description And, naturally, the user may want to be able to see historical data:
- ❖ Previous timers, in a list
- ❖ Their name/description
- ❖ Their total time and whether they were cancelled

Writing A new API: Clockodillo

Assuming you plan to build more than just one device, you need to have some form of identifying the device. For now, assume that each device will send some identifying token, such as a MAC address.

So the user will somehow identify himself with the server, after which all the preceding actions will relate just to a given user ID.

Writing a new API: SECURITY

- ▶ For Clockodillo, perhaps a boss might want to double-check that employees are using the timer. Or a competitor might want to check the descriptions of tasks to spy what your company is working on. Or a (more disreputable) competitor might want to disrupt and discredit the service by entering fake data.

Task	Inputs	Outputs
1. Create a new timed task	User, Timer duration	Timer ID
2. Change duration of timed task	User, Timer ID, New duration	OK
3. Mark timer complete	User, Timer ID	OK
4. Cancel timer	User, Timer ID	OK
5. Describe the timed task	User, Timer ID, Description	OK
6. Get list of timers	User	List of Timer IDs
7. Get information about a timer	User, Timer ID	Description, Create time, Status

Writing a new API: Security

For Clockodillo, you have to consider the risks in sending the identification or authentication data over the Internet-Whether that's a MAC address or username and password.

The two main cases here are as follows:

- Someone who is targeting a specific user and has access to that person's wired or (unencrypted) wireless network. This attacker could read the details and use them (to create fake timers or get information about the user).
- Someone who has access to one of the intermediate nodes. This person won't be targeting a specific device but may be looking to see what unencrypted data passes by, to see what will be a tempting target.

Writing a new API: Security

- ▶ For a web API, you can simply do this by targeting <https://> instead of http://. It does not require any further changes to your application code. It is easy to set up most web servers to serve HTTPS.
- ▶ The O auth 1.0 protocol-used by services such as Twitter to allow third party applications to access your account without requiring your password- is a good example of providing strong authentication without using HTTPS.
- ▶ The content of the API call is still sent in the clear, so An attacker sniffing the network traffic would still be able to see what was happening, but he would not be able to modify or replay the requests.

Writing a New API: Implementing the API

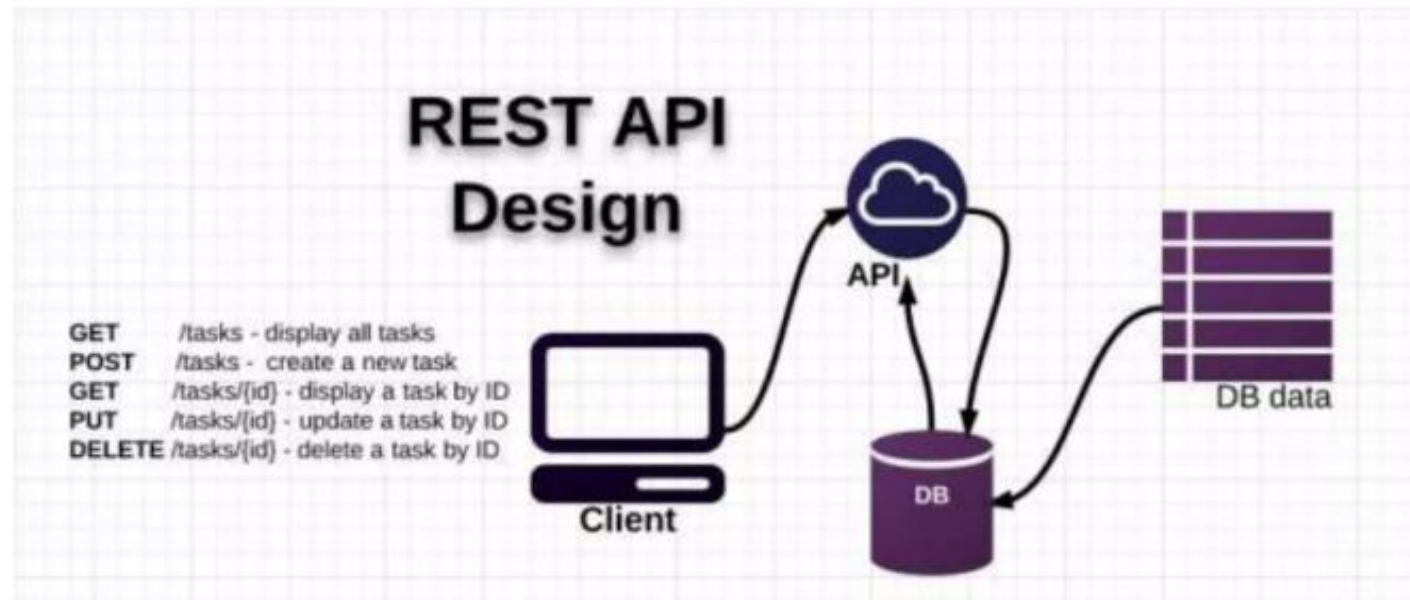
An API defines the messages that are sent from client to server and from server to client. Ultimately, you can send data in whatever format you want, but it is almost always better to use an existing standard because convenient libraries will exist for both client and server to produce and understand the required messages.

Here are a few of the most common standards that you should consider:

- Representational State Transfer (REST): HTTP methods such as GET and POST, but also PUT and DELETE. The result is often XML or JSON but can often depend on the HTTP content-type negotiation mechanisms.

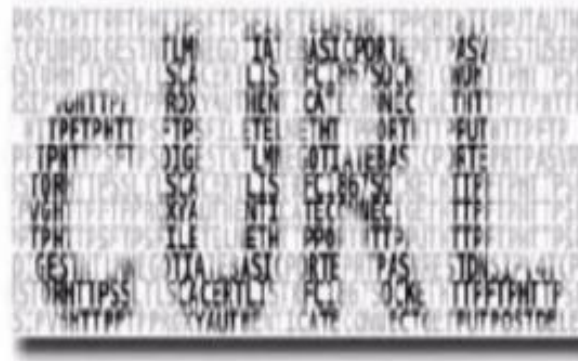
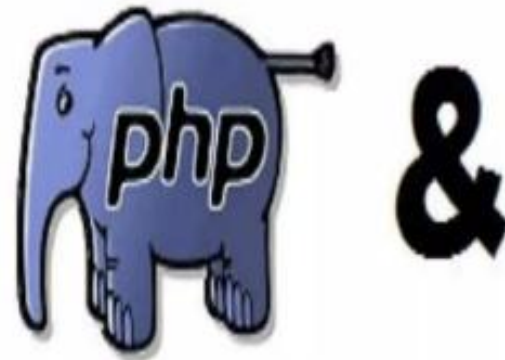
REST

Using REST, you can get the contents of that object using an **HTTP** using some methods like **GET**, **POST**, **PUT**, **DELETE** or **POST** to modify the object.



REST API with cURL

- **cURL** allows you to connect and communicate to many different types of servers with many different types of protocols.
- **cURL** currently supports the **http**, **https**, **ftp** etc.



Using SOAP

- ▶ Easy to consume
- ▶ Rigid - type of checking, provider and consumer have to agree on the exchange format
- ▶ Can use almost any transport to send the request - SMTP, JMS (java messaging service)
- ▶ Asynchronous processing and invocation - guaranteed level of reliability and security then ensure this type of operation
- ▶ Stateful operation - provide support to contextual information and conversational state management.

Using Rest

- ▶ Lightweight - The request and responses can be short
- ▶ Human readable results- Flexible & simple, URIs for identification
- ▶ Easy to build - NO toolkits required
- ▶ Totally stateless operations- stateless CRUD (Create, Read, Update, and Delete) operations.
- ▶ Caching situations - Information can be cached because of the totally stateless operation

Writing a New API: Implementing the API

- JSON-RPC: Access a single web URL like `http://timer`.
- XML-RPC: This standard is just like JSON-RPC but uses XML instead of JSON.
- Simple Object Access Protocol (SOAP): This standard uses XML for transport like XML-RPC but provides additional layers of functionality, which may be useful for very complicated systems.

Writing a New API: Using cURL to test

- ▶ Client for URLs (or cURL) is a software project comprised of two development efforts- cURL and libcurl. Libcurl is a free, client-side URL transfer library with support for a wide range of protocols. cURL is a command-line tool for getting or sending files using URL syntax.
- ▶ While you are developing the API and afterwards, to test it and show it off, you need to have a way to interact with it.
- ▶ Curl simply makes an HTTP request and prints out the result to a terminal.

Writing a New API: Going Further

API Rate Limiting: If the service becomes popular, managing the number of connections to the site becomes critical. Setting a maximum number of calls per day or per hour or per minutes might be useful. You could do this by setting a counter for each period that you want to limit. Then the authentication process could simply increment these counters and fail if the count is above a defined threshold.

Authenticating with Other Services

OAuth to allow hooks to services like Twitter, music discovery site last.fm, etc.

Interaction via HTML

The API currently serializes the output only in JSON, XML, and Text formats. You might also want to connect from a web browser.

Writing a New API: Going Further

Drawbacks: Although web browsers do speak HTTP, they don't commonly communicate in all the methods that we've discussed. In particular, they tend to support the following:

- GET: Used to open pages and click on links to other pages. You can link to `http://timer.roomofthings.com/timers/1234.html` and get the HTML version of the API call (using the “get_timer” template).

- POST: Used when submitting a form or to upload files. To post a timer, you could create a web form like the following:

```
<form method="POST" action="/timers.html">  
<input type="text" name="duration">  
<input type="submit" value="Create a new timer!">  
</form>
```

This form calls the POST action and returns the appropriate HTML

Writing a New API: Going Further

Designing a Web Application for Humans

For example, the following figure shows a static login page, to be served by GET. The API didn't even specify a GET action, as it was superfluous for a computer. This page is entirely for the convenience of a human.

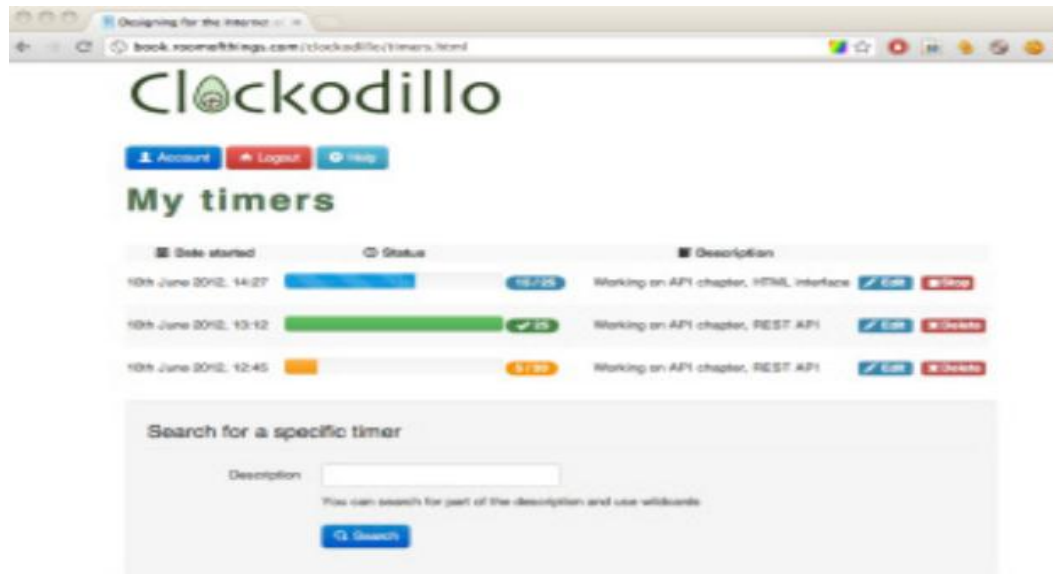


The screenshot shows a web browser window with the address bar displaying "book.roomaffairs.com/clockodillo/login.html". The page features the "Clockodillo" logo, which includes a small green icon of a clock face with a smiley face. Below the logo is the word "Login". A light gray box contains the text "Enter your email address and password". Inside this box, there are two input fields: "Your email address" and "Your password". Below the email field is a small text note: "this is the email you registered with". Below the password field is another small text note: "remember your password is case sensitive!". At the bottom of the form is a green button labeled "Log In".

Writing a New API: Going Further

Designing a Web Application for Humans

The list of timers, instead of being a JSON string containing a raw data structure, is highly formatted. The dates are formatted for humans. The duration of the timer and the status are visualized with colors, progress bars, and a duration “badge”. The page also links to other actions: The “Edit” button opens a page that allows access to the actions that change description, and so on.



Looking at your product from both contrasting perspectives will make it stronger and better designed.

The human-facing list of timers.

Real-Time Reactions

- **Real time reactions**
 - Polling
 - Comet
- **Other Protocols**
 - MQ Telemetry Transport
 - Extensible Messaging and presence Protocol
 - Constrained Application Protocol

REAL-TIME REACTIONS

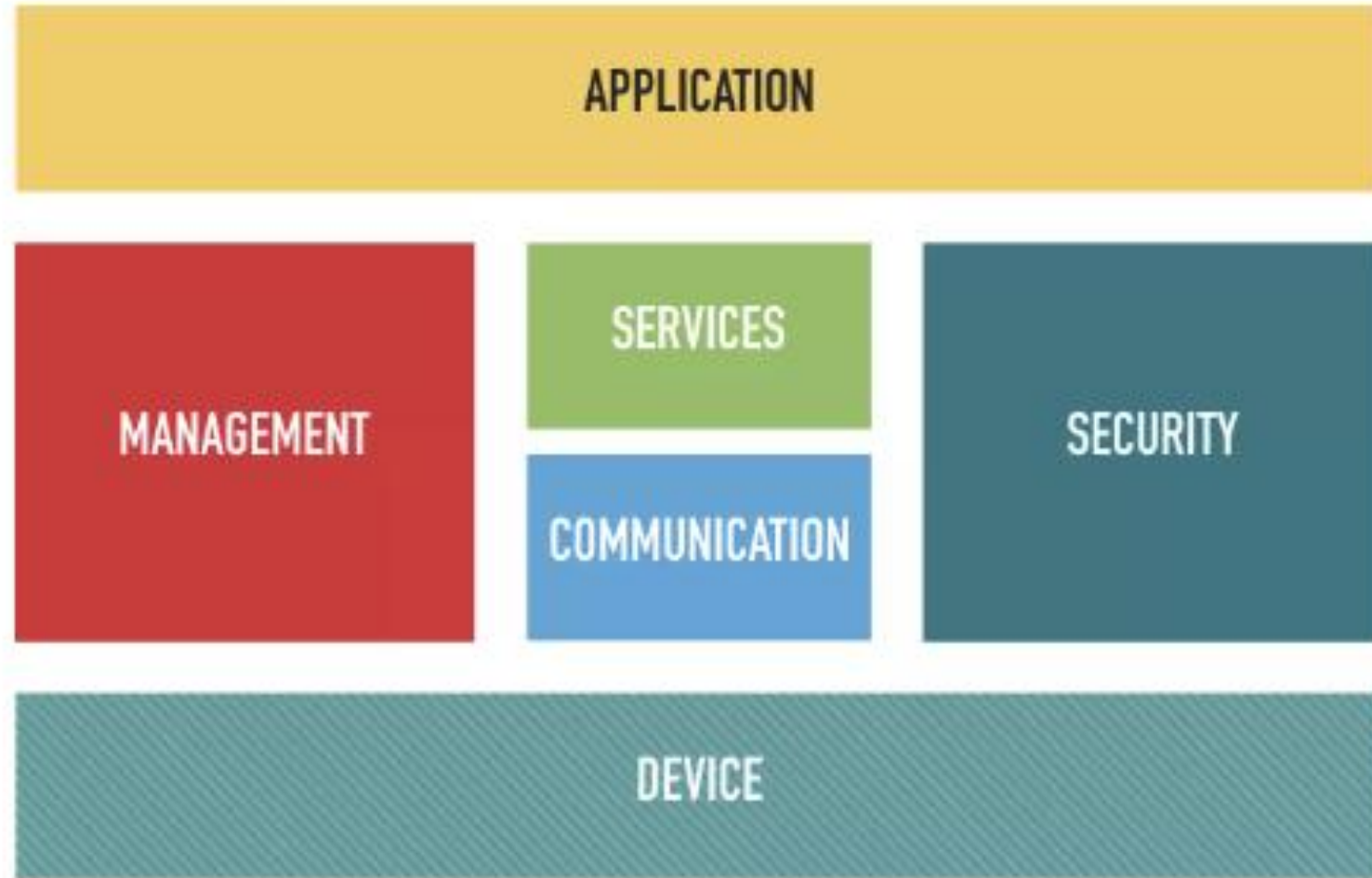
For a bare-bones board such as the Arduino, the current Ethernet/HTTP shields and libraries tend to block during the connection, which means that during that time, the microcontroller can't easily do any other processing.

If you want to perform an action the instant that something happens on your board, you may have to factor in the connection time.

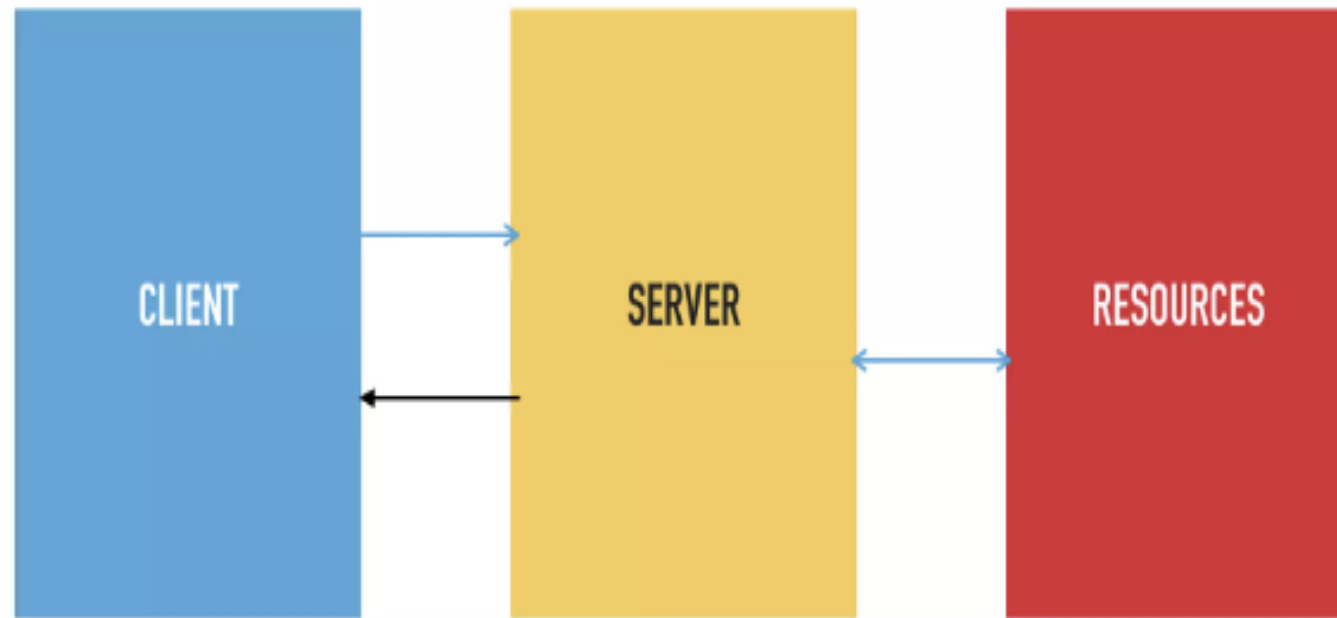
If the server has to perform an action immediately, that “immediately” could be nearly a minute later, depending on the connection time.

We look at two options here: **Polling** and so-called “**Comet**” technologies.

Functional blocks of IoT



Client server model



REAL-TIME REACTIONS: Polling

If you want the device or another client to respond immediately:

Consider these two cases:

- The Where Dial should start to turn to “Work” the moment that the user has checked into his office.
- The moment that the task timer starts, the client on the user’s computer should respond, offering the opportunity to type a description of the task.

REAL-TIME REACTIONS: Polling

The traditional way of handling this situation using HTTP API requests was to make requests at regular intervals. This is called polling.

You might make a call every minute to check whether new data is available for you. However, this means that you can't start to respond until the poll returns. So this might mean a delay to establish the HTTP connection.

You could make this quicker, polling every 10 seconds, for example. But this would put load on the following:

- The server: If the device takes off, and there are thousands of devices, each of them polling regularly, you will have to scale up to that load.
- The client: This is especially important if, as per the earlier Arduino example, the microcontroller blocks during each connect!

REAL-TIME REACTIONS: Comet

Comet is an umbrella name for a set of technologies developed to get around the inefficiencies of polling.

Long Polling (Unidirectional)

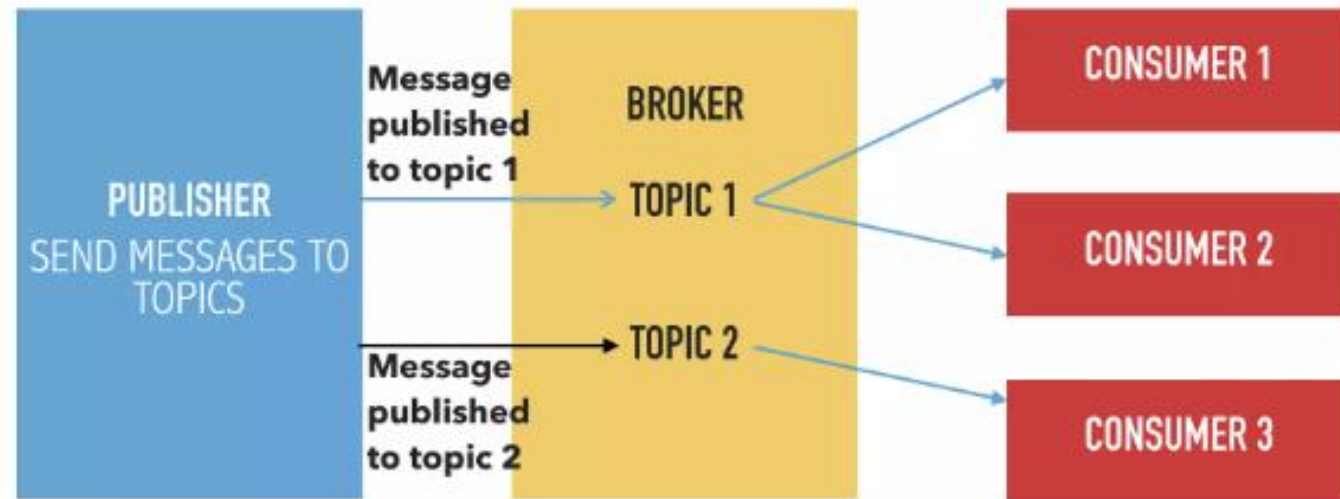
The first important development was “long polling”, which starts off with the client making a polling request as usual. However, unlike a normal poll request, in which the server immediately responds with an answer, even if that answer is “nothing to report”, the long poll waits until there is something to say. This means that the server must regularly send a keep-alive to the client to prevent the Internet of Things device or web page from concluding that the server has simply timed out.

REAL-TIME REACTIONS: Comet

Long polling would be ideal for the case of **WhereDial**: the dial requests to know when the next change of a user's location will be. As soon as WhereDial receives the request, it moves the dial and issues a new long poll request. Of course, if the connection drops (for example, if the server stops sending keep-alive messages), the client can also make a new request.

However, **it isn't ideal** for the task timer, with which you may want to send messages from the timer quickly, as well as receive them from the server. Although you can send a message, you have to establish a connection to do so. Hence, you can think of long polling as unidirectional.

Publish subscriber model



REAL-TIME REACTIONS: Comet

Multipart XMLHttpRequest (MXHR) (Unidirectional) When building web applications, it is common to use a JavaScript API called XMLHttpRequest to communicate with the web server without requiring a full new page load.

Note that XMLHttpRequest is a misnomer because there's no requirement to actually use XML at all. Using this content type is perhaps more sophisticated if you want to be able to receive multiple messages from the server.

In the example of WhereDial, this is unlikely; you're unlikely to change location first to Home and then to Work in quick succession.

However, for an Internet of Things device such as Adrian's Xively meter, which tries to show the state of a Xively feed in real time, being able to respond to changes from the server almost immediately is the essential purpose of the device.

REAL-TIME REACTIONS: Comet

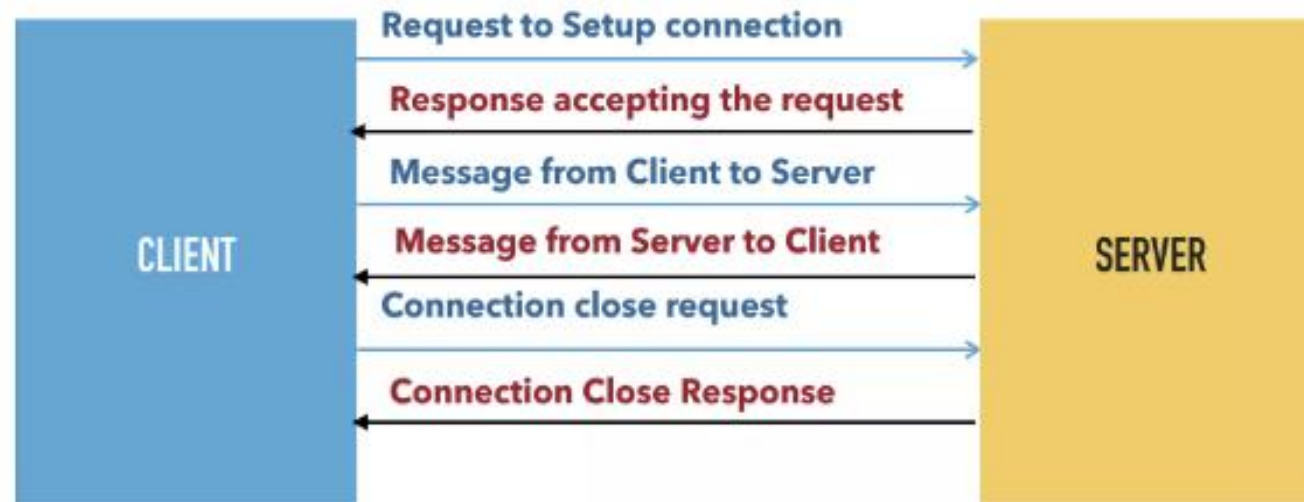
- ▶ HTML5 WebSockets (Bidirectional)

Traditionally, the API used to talk directly to the TCP layer is known as the **sockets API**. When the web community was looking to provide similar capabilities at the HTTP layer, they called the solution WebSockets.

WebSockets have the benefit of being bidirectional.

You can consider them like a full Unix socket handle that the client can write requests to and read responses from.

Exclusive pair communication model



Other Protocols

- ▶ MQ Telemetry transport
- ▶ MQTT is a lightweight messaging protocol, designed specifically for scenarios where network bandwidth is limited or a small code footprint is desired.

It was developed initially by IBM since been published as an open standard, and a number of implementations, both open and closed source, are available, together with libraries for many different languages.

Rather than the client/server model of HTTP, MQTT uses a publish/subscribe mechanism for exchanging messages via a message broker.

Other Protocols

- ▶ MQ Telemetry transport
- ▶ Recipients subscribe to whichever topics interest them and whenever a new message is published on that topic, the message broker delivers it to all interested recipients.
- ▶ This make it much easier to do one-to-many messaging, and also breaks the tight coupling between the client and server that exists in HTTP.
- ▶ A sister protocol, MQTT for sensors (MQTT-S), is also available for extremely constrained platforms or networks where TCP is not available, allowing MQTT's reach to extend to sensor networks such as ZigBee.

References

- ▶ 1. Book - Designing Internet of Things (Chapter 7)
By Adrian Ecewen & Hakim

Question



Thank you

